Midterm 2

Seat: **MIDDLE-B-7**. The second midterm is on April 28 at 10:15am

- Location: DRLB A1 (same location as the first midterm!)
- Time limitation: 80 minutes
- The exam will cover all the material starting from the Replication lecture to the final lecture before the exam
- The exam will be open-book, open-notes
- The exam will have a similar format as in the first midterm

TRUE/FALSE QUESTIONS:

1. In BigTable, read and write to each row is atomic.
    1. True
2. In NFS with client caching, if client A opens a file then client B opens the same file after that, then B is guaranteed to see changes made by A.
    1. False - in NFS with client caching, the data are only pushed when a client closes the file; thus, when B opens the file before A closes the file, B will not see the changes. (Recall: NFS provides close-to-open consistency, and it does not guarantee anything with concurrent write with caching)
3. Every Byzantine fault is a crash fault.
    1. False -- not all Byzantine faults are crash faults; e.g., a malicious change to the value is a Byzantine fault but does not cause crashing.
4. Some Byzantine faults are crash faults.
    1. True - a Byzantine node may decide not to send messages or crash itself.
5. In an asynchronous environment, 3f+1 replicas are required to tolerate f crash-faulty nodes.
    1. False
        1. 3f+1 => Byzantine fault (e.g., in the case of PBFT)
        2. 2f+1 => crash-faulty (e.g., in the case of Paxos)
6. Any independent node fault can be detected by other nodes with sufficient node replicas.
    1. False -- only faults whose effects are visible to other nodes may be detected; faults that are not observable by other nodes cannot be detected by other nodes (e.g., a Byzantine

node that always sends the right messages cannot be detected by other nodes, or a bit flip in the memory of a node is not visible to other nodes).

7. In Paxos, if two nodes propose different values with different proposal numbers, the value associated with the smaller proposal number may still be chosen.

    1. True -- e.g., the smaller-number proposal is seen by a majority number of nodes before those nodes see the higher-numbered proposals.

REPLICATION:

Q1) Prove that under quorum-based replication, if R + W > N then a read will always return the most updated value.

交集保证： 当满足 $R + W > N$ 时，任何由 R 个副本组成的"读集合"与任何由 W 个副本组成的"写集合"必然至少包含**一个共同的副本** 。

In a system with N replicas, a write operation is only successful if at least W replicas successfully perform the write. Thus, at most N - W replicas are not aware of the most recently written value. Since a read is only successful if the client receives responses from at least R nodes, and because R + W > N or R >= (N - W) + 1, at least one of those nodes will be aware of the most recent write. Thus, the read operation will always give the most updated value.

Q2) In your own words, describe one advantage and one disadvantage of quorum-based replication compared to primary-based replication.

- Advantages: more flexible; support tuning of the W/R values based on the performance requirement; no single point of failure (if we select N,W < N).
- Disadvantage: much more complex to implement; not easy to extend for cases where the system must provide read and write even if only one replica is alive (e.g., if W nodes fail in quorum-based replication, we have to change the replica size and quorum size dynamically, and that is not as straightforward to implement).

Q3) For replication to work in general, it is often necessary that all operations be carried out in the same order at each replica. Give an example scenario in which this ordering is not necessary.

An example is the case where the operations are read only, then their order does not matter.

Q4) How do you set W, R, N in the following cases:

- Data must be strongly consistent; minimize the loss of latest data under multiple failures; Reads should be as fast as possible.

  R = 1, W = N, and N is as large as possible.

    - Data must be strongly consistent ==> we must have R+W>N and W+W>N to avoid read-write and write-write conflicts.
    - Since we want to minimize the loss of **latest** data under multiple failures, we want W = N and N to be as large as possible.
    - Since W = N and reads should be as fast as possible, so we can select R to be the minimum possible that satisfy R+W>N, thus R = 1.
- Eventual consistency is okay; reads/writes should be as fast as possible; up to 2 failures should not cause system become unavailable.

  R = W = 1, N = 3.
    - Eventual consistency is okay, so W and R are **not** subject to read-write or write-write constraints. Reads and writes should be as fast as possible, so we can set R and W to be as small as possible, i.e., R = W = 1.
    - Up to 2 failures should not cause the system to be unavailable, we have N = 3.

BIGTABLE:

In your own words, explain:

Q1) How Bigtable can support real-time performance while still ensuring fault tolerance? What is the implicit assumption about the software?

Bigtable performs the requests in memory, so it gets good performance. It achieves fault tolerance by doing logging of write requests and checkpointing, and the log and checkpoint files can be used to recover the tablets in case a node fails.

The assumption here is that software is deterministic; otherwise, replaying will not work and thus recovery with replaying won't be possible.

Q2) How scalability is achieved in Bigtable?

Scalability is achieved by Bigtable's distributed architecture that spreads data across multiple tablet servers. Each tablet server handles read/writes for a subset of data. As more data is added to the system, Bigtable hashes and dynamically load balances data between tablet servers using row keys (each tablet shards a range of rows). More servers can be added to the system to increase the capacity horizontally, and we can increase the number of tablet servers accordingly in that case.

FAULT TOLERANCE:

Q1) In your own words, explain why 3PC is not safe in the presence of network partition?

Q1) In your own words, explain why 3PC is not safe in the presence of network partition.

In the presence of a network partition, we will have different groups of nodes that can communicate with each other, but not with other groups. It is then possible for these groups to come to differing consensuses in 3PC. For example, in group 1, no node has received a PRECOMMIT or COMMIT, so this group will abort. In contrast, in group 2, a node has received at least a PRECOMMIT/COMMIT, so these nodes will commit. (This scenario can happen when all of the k subordinates to which the coordinator sent the PRECOMMIT message fall into group 2.) Thus, atomicity is violated (i.e., 3PC is not safe).

Q2) In 3PC, if all nodes voted Yes, the coordinator sends PRECOMMIT message to at least k subordinates and wait for their acknowledgements before sending out the COMMIT message. Assuming that there is no network partition, what is the minimum value of k to ensure that the protocol remains safe even if there are 3 node faults?

If there are 3 node faults, k needs to be at least 3.

- If the coordinator is not faulty, each alive node can always talk to the coordinator to obtain the correct decision and everyone will arrive at the same outcome. Hence, all is well.
- If the coordinator is also faulty, then at most 2 subordinates can fail (since we have at most 3 faults). To ensure at least one node that receives PRECOMMIT survives, we need k > 2, i.e., k = 3. (*Suppose instead that k = 2, and that the coordinator send COMMIT after two nodes acked its PRECOMMIT, and that the COMMIT reached these two nodes but the coordinator failed before it sends to anyone else. Then, it's possible that those two nodes committed their parts, but they then failed too. The remaining subordinators have not seen any PRECOMMIT or COMMIT, so they abort. Thus, atomicity is violated*.)

Q3) In centralized checkpoint, why does a process need to queue outgoing messages until it receives DONE from coordinator?

Queuing outgoing messages are necessary to avoid inconsistent snapshot. As messages may take different amount of time over the network, the following scenario is possible if nodes don't queue messages until receiving DONE from the coordinator: Node A sends an outgoing message m after it has performed a checkpoint, and m arrives at node B before the CHECKPOINT message from the coordinator. Then, node B's checkpoint would contain the receipt of m. In other words, the receipt of m is in the snapshot but the sending of m is not (because it is sent after A's checkpoint). This leads to an inconsistent snapshot.

Q4) The Chandy-Lamport distributed snapshot algorithm requires FIFO ordering of messages between any two processes. Using an example, explain why the lack of FIFO ordering will result in an inconsistent snapshot.

Suppose A initiates a snapshot, and it performs its checkpoint and sends the marker along

outgoing channels. Then, A sends another message m to node B (recall that nodes do not need to queue outgoing messages after their checkpointing in Chandy Lamport).

Without FIFO, the message m may arrive at B **before** the marker. Thus, the receipt of m is in B's checkpoint (because B only does checkpoint when it receives the first marker) but the sending of m at A is not in A's checkpoint. This leads to an inconsistent snapshot.

Q5) Explain how a node knows that the Chandy-Lamport algorithm has been completed. For a group of N nodes, what is the complexity of the algorithm in terms of the number of messages being sent between nodes (in big O notation)?

A node knows that the algorithm is completed after it has received a marker from every other node. Since each node needs to send N-1 markers, where N is the number of nodes, we have a complexity of O(N^2) in big O notation.

STATE MACHINE REPLICATION:

Q1) In your own words, explain why Paxos cannot tolerate f failures with fewer than 2f+1 nodes.

Briefly, when n <= 2f, there will not be a majority if f nodes fail, and thus the algorithm cannot proceed. The node can try to abandon a proposal, but it cannot decide on a value in the presence of f failures.

Q2) In the Paxos algorithm, briefly explain why a proposer must wait for acknowledgements to its prepare message from a majority of nodes before it can move to the accept phase.

Suppose to the contrary that nodes do not wait for ACKs from a majority of nodes. Then, we can show that it is possible for

- two proposers to issue ACCEPTs with different values, and
- these ACCEPTs will be accepted by a majority of nodes, which will in turn lead to two different values being decided.

Consider, for example, a system with nodes A to Z. Let's say nodes do not wait for ACKs from a majority but send accept messages as soon as they receive one ack, then we can have:

- A sends PREPARE(1) to one node C, receives back ACK(1,-,-).
- B sends PREPARE(2) to a different node D, receives back ACK(2,-,-).
- A then issues ACCEPT(1, X) and sends to C to Z. Since all except for D have not acked a higher proposal number before, they all accept the ACCEPT(1,X). Since there is enough majority of

ACCEPT(1,X), we can see that X is decided.

- B then issues ACCEPT(2, Y) and sends to C to Z. Since no one has acked a higher proposal number than 2, they will all accept the ACCEPT(2, Y). Since there is enough majority of ACCEPT(2,Y), we now see Y is decided.

Thus, two conflicting values are decided. (You can construct a similar example where a node issues accept after some k > 1 acks, where k is not a majority)

Q3) Paxos guarantees safety but not liveness. Using an example, show a scenario in which the protocol for a (given) entry never terminates (i.e., no value ever be chosen for the entry).

Consider a system with 5 nodes A, B, C, D, E. The following scenario shows a case where the algorithm never terminates:

- A issues PREPARE(1) to C, D, E. These nodes will respond with ACK(1,-,-).
- B issues PREPARE(2) to C, D, E. These nodes will respond with ACK(2,-,-).
- Since A receives ACKs from a majority, it issues ACCEPT(1,X) where X is its proposed value, and send the accept message to C,D,E. Because C,D,E already acked a prepare message with higher number, they do nothing. After a timeout, A issues a new PREPARE(3) and send to C-E, who will respond with ACK(3,-,-).
- Since B receives ACKs from a majority, it issues ACCEPT(2,Y) where Y is its proposed value, and send the accept message to C,D,E. However, as these nodes saw a prepare with higher number (number 3), they ignore the accept message from B. After a timeout, B issues PREPARE(4) and send to C-E. They send back ACK(4,-,-).

- A issues ACCEPT(3,X) and send to C-E, which will be ignored. A retries later with PREPARE(5).
- ....

The above keeps going on forever and no value will ever be decided.

NON-CRASH FAULT TOLERANCE:

Q1) In the Byzantine Generals solution with signature, explain why it is safe to stop forwarding the order if the order already has f endorsements?

We want that if a loyal general receives a valid order O, then every other loyal general will also receive O. If there are f endorsements of an order, at least one of these endorsements will be from a loyal general, and that loyal general must have forwarded the order to every other general as well. This implies that if a loyal general k receives a valid order O, then any other loyal general also receive the order O. In other words, the set of valid orders that each loyal general receives is identical, which means that by applying the same deterministic function to decide on the

identical, which means that by applying the same deterministic function to decide on the decision, they all end up with the same decision. Therefore, it is not necessary to forward further.

Q2) In your own words, explain why at least 3f + 1 is required to tolerate f Byzantine faults in an asynchronous setting?

First observe that, in any working protocol, a replica must be able to proceed after communicating with **n - f** replicas, since f replicas might be faulty and not responding.

Let's consider a scenario where a node receives responses from **a subset S of only N - f** nodes, but not from the remaining nodes. In an asynchronous system, since message delay is not bounded, it is possible that the nodes without responses are actually correct nodes and did respond, but their messages simply took infinitely long to arrive. In that case, the faulty nodes must belong to the set S. Since up to f nodes may be faulty, the number of correct responses can be as low as **(N-f) - f = N-2f**. To have a correct majority, we must have:

N-2f (#correct responses) > f (#incorrect responses), or N > 3f.

Q3) Alice claims that she has found a way to detect observable Byzantine node faults in asynchronous setting using only f+1 replicas (e.g., if some node fails, another node can detect it). Do you believe her? Explain. (Here, an observable faulty node refers to the case where the node's faulty behavior is visible/exposed to another node, e.g., through its messages or lack of messages. If a faulty node only does what a correct node would do, then the fault is not observable by other nodes.)

Yes! Since at most f nodes are Byzantine, at least one node is correct. That correct node can compare its own values with the values produced by each other node. Any discrepancies indicate that the other node is faulty and is detected by the correct node. *(Note that we implicitly assume here that the fault is observable, i.e., the faulty node does not always send correct messages; see True/False Q6 above.)*

DISTRIBUTED FILE SYSTEMS:

Q1) What calling semantics below does RPC2 provide in the presence of failures?

1. a) Exactly once
2. **b) At most once**
    1. With RPC2, a client can tell when the server disconnects, in which case it will just go to the Emulation state. However, this doesn't guarantee that the update will be reflected on the server. Hence, the semantics is at-most-once.
3. c) At least once

Q2) Explain how Coda prevents read-write conflicts on a file that is shared between multiple

readers and a writer.

Coda does this by inferring from the session type whether a user is a reader or writer. For the single writer, it acquires the exclusive lock to write, and once an exclusive lock is given to a client, no other writer can obtain the lock.

Note that the above does not prevent the other users who have previously obtained a shared lock on the file (i.e., before the first writer acquires the exclusive lock) from reading this file. When a reader reads a file that is currently being written to, it knows that it is reading this outdated copy because of the invalidation message. This is okay because the effect is the same as if the read transaction has occurred before the write transaction.

Q3) Name one advantages and one disadvantage of the remote access model compared to the upload/download model. Which model does NFS follow?

An advantage of the remote access model is that it is easier to enforce strong consistency, since all operations take place on the server side and the server can always synchronize any concurrent updates. This is not easily done with upload/download model, since the update is done first by the client locally.

A disadvantage of the remote access model is performance, since you need an RPC for every request which is very time consuming (due to network delay).

GFS

Q1) In your own words, explain how GFS ensures that there is at most one primary for a given chunk at any point in time.

In GFS, the master coordinates who is the primary for a given chunk through leases. The master gives one of the replicas a lease that comes with a timeout. For the period of that lease, that replica is the primary. The master is the sole arbitrator of this - no other replica is allowed the lease. The master can change what replica has the lease; however, it does this by revoking the old lease, or by granting a new lease when the previous has expired. This ensures that there is only at most 1 lease, i.e., one primary, at a time.

Q2) In GFS, it is possible for two replicas of the same chunk are not byte-wise identical. Provide a scenario under which this occurs.

GFS has a relaxed consistency model (eventual consistency), and the inconsistency may occur during a partially failed write for example. Say a webpage is being recorded in GFS, it may successfully be written to the primary, but the write operation may fail on the second or third replica. In this case, the replicas may diverge.

Map Reduce

MapReduce

Q1) Consider a social network graph, where each node of the graph represents a user, and each (undirected) edge connecting two nodes indicates that the corresponding users are friends. Such a graph can be written as a sequence of key-value pairs, where the key represents a user and the value is a list of his/her friends. For example,

```
(Alice, [Bob, Charlie, Elizabeth])
(Bob, [Charlie, Dorris, Sam])
...
```

Write in pseudocode a MapReduce program to find the list of mutual friends between any pair of users. The output should be tuples of the following form:

```
( user1, user2, [list of common friends between user1 and user2])
```

Explain the input and output key-value pair in the map and reduce function.

```
map(k,v){
    // key k = user; value v  = list of fiends
    for i = 0 : length(v) - 2
        for j = i+1 to length(v) - 1
            out_key = (v[i], v[j])
            out_value = k
            emit(output_key, output_value)
}

reduce(k,v){
    // k = a pair of friends; v = list of their fiends
    emit(k,v)
}
```

Q2) Design a MapReduce program to compute the maximum number in a list of N input files, where each file contains multiple numbers, one per line (that is, similar format to the input data for HW1). Specifically, please describe

- What is the input and output of the Map function?

  Map function:

    - Input: <a file name, the file content> (We can also use a range of lines in a file instead; like in word count)
    - Output: : Emits a fixed key (we don't really care what this is, say "max") and value that is the maximum number found in the file.
- What is the input and output of the Reduce function?
    - Input: <"max", a list of maximum numbers from each file>.
    - Output: <"max", the max value of the list of values in the input)
- In your own words, give two reasons for why MapReduce is not suitable for iterative applications.
    - MapReduce writes all intermediate key-value pairs to disk, which requires a lot of read/write from disk, so it has very low latency and not suitable for iterative applications.
    - Iterative applications also often involve many rounds of Map/Reduce phases, which makes the inherent performance problem in MapReduce even more severe.

Spark

Bob is working on his CIS 5550 final project, for which his team needs to build a web search engine. He is responsible for implementing the PageRank algorithm to rank the pages. For this, he decided to reuse the Spark code for the PageRank example studied in class, as shown below.

```
val links = spark.textFile(...).map(...).persist()
var ranks = ... // RDD of (URL, initialRank) pairs
for (i <- 1 to ITERATIONS) {
  val contribs = links.join(ranks).flatMap {
  (url, (links, rank)) =>
       links.map(dest => (dest, rank/links.size))

 }
 ranks = contribs.reduceByKey((x,y) => x+y)
                 .mapValues(sum => a/N + (1-a)*sum)
}
ranks.save("out.txt")
```

However, Bob discovers that the algorithm takes many hours to run on his (very large) collection of crawled pages. Curious, Bob decided to add some debugging code into the program to print out how much each line of the "for" loop (from Lines 4-10) takes. The debug output shows that each line takes only a couple of milliseconds to execute.

Could you explain why the whole program takes so much time despite each loop iteration only takes only milliseconds? You may assume that ITERATIONS is set to 100.

The code in the loop applies Spark transformations to the RDD, so Spark simply builds a lineage graph of operations that are applied to the links RDD at each stage of the iteration. However, these transformations do not actually trigger the work: this entire stack of operations are only performed when a Spark action -- the save() action -- is called. Therefore, output collection to out.txt (line 12) is what dominates the execution time.

DHT/DYNAMO

Q1) In your own words, explain the CAP theorem.

The CAP theorem states that it is not possible to achieve consistency, availability and partition tolerance altogether, and that only at most two of them can be achieved at the same time.

Q2) In your own words, describe how BigTable and Dynamo make their design decisions to work around the impossibility result proven by the CAP theorem?

- **BigTable (倾向于 CP)**：BigTable 优先考虑一致性 。它通过单一主节点（Master）分配任务，并确保对单行的读写是原子性的 。当发生网络分区时，受影响的分片可能会暂时不可用，以防止产生不一致的数据 。
- **Dynamo (倾向于 AP)**：Dynamo 优先考虑高可用性 。它允许客户端在发生分区或节点故障时继续执行写操作（如购物车添加商品）。为了解决由此产生的冲突，它使用了**最终一致性 (Eventual Consistency)**，通过向量时钟（Vector Clocks）记录版本，并在随后读取或合并时解决冲突 。
- BigTable sacrifices availability for consistency and partition tolerance--in the presence of network partitions, it rejects writes and thus may violate availability.
- Dynamo prioritizes availability and partition tolerance over consistency--in the presence of network partitions, it allows writes to succeed even if replicas cannot communicate with one another, thus it may lead to inconsistent state; it will later try to reconcile any inconsistencies using vector clocks.

Q3) In Dynamo, every node knows every other nodes. Name one advantage and one disadvantage of this approach. Why does this choice make sense for Dynamo?

- **优点**：路由速度极快，任何键的查询都可以通过“一跳（Zero-hop/One-hop）”直接定位到目标节点，消除了多级跳转的延迟 。
- **缺点**：扩展性受限。当节点数量增加到数万个时，维护全局成员列表的心跳开销和内存占用会变得非常庞大

非常庞大 。

- **合理性**：Dynamo 设计用于**数据中心内部**（如亚马逊的内部基础设施），节点数量相对稳定且受控（通常在数百到数千级），这种规模下，完全路由带来的性能收益远大于其维护成本 。
- Pros: Efficient lookup -- since every node knows every other node, it takes O(1) to find the data.
- Cons: High state management overheads -- as nodes join and leave the system, we need to update every node's information about other nodes.

- This choice makes sense for Dynamo because it operates in data center, so nodes do not join and leave all the time (as in a case of peer-to-peer systems, e.g., when nodes are users' devices themselves).

Q4) In DHT, what is the core tradeoffs between having more connections (number of nodes that each node knows about) vs. fewer connections?

- **连接较多（如 Dynamo）**：查询速度快（跳转次数少），但维护路由表的成本（心跳检测、节点加入/离开时的更新）非常高 。
- **连接较少（如 Chord）**：维护成本低，每个节点只需记录少量邻居（$O(\log N)$），能支持数百万级的海量节点；但查询延迟较高，需要经过多次跳转（$O(\log N)$ 步）才能找到数据 。

More connections will make lookup more efficient (less hops) but it also increases the complexity of the state management when nodes join/leave the system.

Q5) In class, we have studied multiple distributed storage systems, including NFS, GFS, Coda, Chord, Dynamo, Bigtable. In your own words, discuss for each system

- which scale does each support?
- what kinds of workloads each was designed for?
- what level of consistency each provides?

Here are some characteristics of the different systems. Note that they are at different levels: Bigtable and Dynamo are KVS (roughly highest level of abstraction among these); Chord is a DHT (you could view DHT as a middle level of abstraction); and NFS/Coda/GFS are distributed file systems (lowest level of abstraction). For example, Bigtable uses GFS under the hood, and Dynamo uses DHT under the hood).

- NFS:
    - **Small scale**, single or a few servers; no replication
    - **Support POSIX style operations**, with **diverse read/write operations**

    - **Close-to-open** consistency; no guarantees on concurrent writes; separate lock service for mutual exclusion
- Coda
    - **Larger scale** than NFS (thousands of nodes)
    - Support general/standard file operations
    - **Optimistic replication** (read-one, write-all that can be reached)
    - Designed for **disconnect operation** (offline accesses); automatic system-level reconciliation using vector clocks
    - **Weak transactional semantics** (effect is the same as if the transactions had been serialized)
- GFS
    - **Very large scale** (data center); very large capacity; extremely **large files**
    - Designed for **sequential reads/writes**, heavy appends; batch operations
    - No support for offline accesses
    - **Primary-based replication** (read from any, write to all through primary)
    - **Relaxed consistency** (eventual consistency); reject writes if network partitions among replicas but no 2PC or 3PC
- Bigtable
    - **Very large scale**, with replication via GFS
    - Designed for **structured data**; sparse tablets; row keys sorted lexicographically
    - Support **diverse workload requirements** in data item sizes, latency requirements, etc.
    - Atomic per-row operations
- Dynamo
    - **High write availability** (always writable, always available)
    - Designed for **very fast performance** (low latency)
    - **Consistent hashing** of keys, but **each node knows all other nodes** (for data center setting/not peer to peer).
    - Sloppy quorum with hinted handoff (not enforcing W+R>N); **eventual consistency**
    - **Objects are typically small in size** (unlike GFS where each chunk is very big, and unlike Bigtable where objects can be of diversed sizes).
- Chord

- Chord
    - Designed for **peer to peer systems** in which nodes join and leave frequently
    - **Consistent hashing with O(log N) trick** -- each node knows O(log N) nodes; routing takes O(log N) hops

Q6) Which distributed system is best fit for each of the following application scenarios?

1. The Census Bureau publishes census microdata for researchers. The data is stored based on region, then by household, then by person.

   Bigtable -- we need systems for highly structured data and very large scale

2. A startup is building a decentralized social bookmarking service where each user stores tags and URLs on their own device, and the system must locate any tag efficiently as users constantly connect to and disconnect from the system.

   Chord -- this is a peer to peer system where nodes join and leave constantly

3. A small film-production company uses local office machines to share scripts, schedules, and administrative files over a highly reliable LAN.

   NFS -- something small is sufficient; no need for offline access

4. A global multiplayer game stores per-player session state, which must accept writes during cross-region network partitions.

   Dynamo -- we need a large scale system that is highly available even under network partitions.